

# Web Cache Overflow: Exploiting Imprecise Keys for Cache Degradation and Beyond

Matteo Golinelli  
University of Trento

Kaan Onarlioglu  
Akamai & Northeastern University

Bruno Crispo  
University of Trento

## Abstract

Web caches support the scalability needs of contemporary web applications by storing frequently accessed objects closer to clients. Web caches are conceptually associative arrays, tracking stored objects using cache keys consisting of HTTP request fields. However, these cache keys are often imprecisely defined by website operators. This allows clients to craft a multitude of requests that target the same object, but map to different cache keys.

In this work, we show that request elements included unnecessarily in cache keys can be abused to create redundant cache entries. In susceptible deployments, sustained generation of such redundant copies reduces cache effectiveness and increases origin load, facilitating eviction-dependent attacks. Our experiments reproduce cache degradation across five stand-alone caching proxies and characterize how these parameters affect attacker cost and cache hit rate, potentially resulting in denial-of-service attacks. We conclude that precise cache-key design is the most direct mitigation against this abuse vector and should be recognized as a security best practice.

## 1 Introduction

Web caches are reverse proxies that sit between clients and origin servers, storing copies of frequently accessed objects. Web caches have reached existential importance in meeting the low access latency requirements for clients and traffic offload demands for origins. Cache degradation can rapidly escalate to costly network congestion and denial-of-service (DoS) incidents. Web caches are ubiquitous in contemporary web architectures, and their efficient utilization is crucial.

More generally, the topic of caching has been a staple of computer science. Researchers have explored various caching strategies, cache replacement algorithms, and even adversarial techniques to impair cache efficacy or launch cache-based side-channel attacks for different hardware and software engineering contexts. However, web caches remain

particularly susceptible to abuse due to two important factors: their immense exposure to *untrusted input* that influences the caching decisions, and widespread web cache configuration anti-patterns that do not account for such potential abuse.

Cache stores can be characterized as an associative array abstract data type; they associate every cacheable object with a *cache key*. Cache keys are n-tuples that may consist of any data element included in an HTTP request according to the website operator's needs, such as the request line components, header values, or structured data fields inside the body. Upon receiving a request that matches a caching rule, the web cache derives the corresponding cache key and looks up the object in the cache store using that key. If the object exists, the request is immediately served from the cache. Otherwise, if the cache key does not correspond to an existing cache store entry, the web cache forwards the request to the origin server, receives a fresh copy of the requested object in the corresponding response, and then caches it under the said key.

In practice, however, cache keys are often not *precisely* defined. For example, a website operator who creates a caching rule for a static image object served from "example.com/pic.jpg" may simply define the cache key as the full URL, overlooking the possibility of query strings included in the URL. This oversight would result in a subsequent request for a functionally equivalent but syntactically different URL, such as "example.com/pic.jpg?junk=123", yielding a different cache key, because the two URLs do not exactly match. If discovered, a client can then take advantage of this situation to craft a multitude of requests for the same object, but with different cache keys, causing the web cache to retrieve the object from the origin anew every time. This is an instance of a well-known trick called *cache busting*, often used by web developers for testing application changes, without needing to purge caches to observe the effects of their changes.

Motivated by these observations that clients influence the HTTP requests used to derive cache keys, and that imprecise cache key definitions occur in practice, we hypothesize that attackers can systematically weaponize cache-busting

techniques to create redundant cache entries. In susceptible deployments, sustained generation of such entries reduces cache effectiveness by forcefully evicting useful content. We call this attack Web Cache Overflow (WCO).

One obvious impact of WCO is increased origin load, which could lead to service degradation and escalate to a DoS. Moreover, since web caches are frequently deployed in shared cloud infrastructures and hosting providers, an attack resulting from one tenant’s imprecise cache keys may DoS all tenants of the platform. However, attackers can also leverage the capability to purge popular cached objects for other nefarious ends. For example, *cache poisoning* attacks that aim to trick caches into storing malicious content are on the rise. Such attacks are challenging to launch against frequently accessed objects (e.g., a JavaScript file embedded on the home page), as such objects are often perpetually cached due to their popularity. With WCO, an attacker could fill the cache to trigger purging of even popular objects, exposing a window of opportunity to poison that object’s now unused cache key with malicious content.

A novel characteristic of WCO is that, unlike previous cache attacks in the literature, generating redundant entries does not require prior knowledge of the replacement algorithm or the popularity of legitimate objects. Instead, it is an attack enabled by the fundamental workings of HTTP and web caches; it can be exploited by any unauthenticated, unauthorized, unsophisticated Internet client, and it is difficult to mitigate without incurring high costs to website operators.

We test these claims through a detailed evaluation of WCO. We first reproduce cache degradation across five popular caching proxies, and then perform focused tests using NGINX to explore the influence of cache capacity and object size. Testing WCO against production websites is not ethically feasible, as DoS attacks are disallowed in bug bounty programs. In lieu of penetration testing, we conduct a measurement study on the Tranco top 10k domain list to identify equivalent objects cached under different cache keys, the prerequisite condition for the attack.

We finally discuss seemingly intuitive mitigations, namely cache store quotas, request rate limiting, and object deduplication, including their deployment tradeoffs. Our experiments show that, in realistic scenarios, these mitigations may be ineffective, counter-productive, or unreasonably costly. We conclude that addressing the prerequisite condition by defining *precise* cache keys is the most direct defense, and we document this as a security best practice for the first time.

To summarize, we make the following contributions.

- We show for the first time that imprecise cache keys can be abused to create redundant cache entries and degrade cache effectiveness in susceptible web cache deployments.
- We present WCO, a cache pollution attack that increases eviction pressure on legitimate cached objects by sustaining redundant entries.
- We evaluate the parameters that influence WCO’s feasibility and efficacy, characterizing its origin-load consequences and other potential implications, such as assisting cache-poisoning attacks.
- We present possible mitigations and evaluate them, concluding that defining precise cache keys is the preferred method rather than workarounds.
- We release an open-source penetration testing tool to assist operators in automatically detecting imprecise cache keys on their websites.

**Availability** We release two open-source tools: a penetration testing utility for detecting imprecise cache keys, intended for website operators and security testers, and an implementation of our attack to evaluate systems against WCO. All the code, data, and artifacts required to reproduce our experiments are also available in our repository at <https://github.com/Golim/web-cache-overflow>. The repository includes several README files with details about the repository contents and instructions for reproducing our experiments.

## 2 Background & Related Work

### 2.1 Web Caches

Websites are rapidly growing in size due to an increasing number of objects necessary to render each page, such as images, videos, fonts, JavaScript files, and style sheets [22]. Furthermore, latency-sensitive media streaming services and bandwidth-hungry big file downloads have become commonplace. All of these can put an enormous strain on an origin server’s network resources if left unaddressed. Therefore, web caches have become an essential element of the modern web.

Web caches are positioned between a client and an origin server to temporarily store frequently accessed objects, preventing unnecessary repeated data transmissions and reducing both bandwidth and latency. Multiple caches can be present on the delivery path, starting with client-side caches (e.g., implemented inside a web browser), layers of reverse proxy servers acting as intermediary caches, and finally caches co-located with the origin server, forming cache hierarchies. Content Delivery Networks (CDNs) that provide performance and security services over distributed reverse proxy networks similarly cache content as a core capability.

In this paper, we use the term *web cache*, or simply *cache*, to refer to server-side web caches. We leave client-side caches out of scope, as attacks on such caches (i.e., a client attacking itself) do not constitute a valid threat model. We also do not

explore niche uses of private caches, but instead focus our discussion on the far more prevalent public caches that have become a crucial part of web architectures.

## 2.2 Cache Control

HTTP provides a standard mechanism for origin servers to signal to the caches on the path whether a response may be stored, via the `Cache-Control` response header. The directives that can be included with this header define the cacheability of an object, its expiration time, and the revalidation actions.

RFC 9111 states that caches *MUST* respect `Cache-Control` directives. However, this is not the case in practice, as also documented by prior academic work (e.g., [16]). Website structures and caching policies may frequently change, which makes maintenance of `Cache-Control` directives to reflect those changes an operational burden for website operators, especially in enterprise infrastructures. Therefore, popular cache technologies offer configuration options to disregard `Cache-Control` headers, but instead implement *cache rules* centrally at the cache. Cache rules can be made as general or specific as needed, allowing website operators to craft them via domain-specific languages or regular expression matches on virtually any part of a request or its corresponding response (e.g., [5, 24]).

Once a response is determined to be cacheable, the cache derives a *cache key* for the respective stored object. The cache key acts as a unique index into the cache store, enabling the cache to easily check whether it already holds a copy of the object upon receiving subsequent requests for the same resource. That is, the cache store implements an associative array as an abstract data type, keyed with the cache key.

Cache keys are n-tuples (or hashes of those n-tuples) that may consist of any element of an HTTP request, including the method, path, query string parameters, headers and their values, or structured data fields in the body payload. Request elements that are included in the cache key are called *keyed*, and the rest *unkeyed*. A typical default cache key is a simple 2-tuple consisting of the full URL (e.g., [23]), but additional elements may also be keyed for more complex situations, for instance, the `Origin` header for CORS support (e.g., [4]). Website operators customize cache keys for their needs.

A *cache purge* is the process of invalidating a cached object, meaning that the next request that matches that object's cache key will be a cache miss. That request will then be forwarded to the origin server, and the cache will store the new response under the same cache key. Web caches provide secure mechanisms for website operators to purge caches, typically through an authenticated web interface or an API. Arbitrary web clients should never be able to purge the cache, as this would facilitate DoS attacks, cache poisoning, and cache-timing-based side-channels.

## 2.3 Cache Busting

*Cache busting* collectively refers to client-side techniques used for intentionally bypassing a cache and receiving a fresh copy of the requested object from the origin server. Cache busting is frequently used in web development to rapidly test website changes without needing to purge any intermediary caches (which may not even be under the developers' control) that may still hold stale copies of modified resources. Similarly, penetration testers often leverage cache busting to avoid poisoning cacheable endpoints with attack payloads.

According to RFC 9111, clients may attempt cache busting by including a `Cache-Control` header set to the "no-cache" directive in their requests [8]. However, caches are not required to honor this request directive, and many production infrastructures in fact do not, due to the aforementioned security implications. Instead, practical cache busting techniques involve identifying keyed elements of a request that do not alter the response, and intentionally modifying those elements in requests to trigger a cache miss.

## 2.4 Cache Status Headers

Web caches often add cache status headers to responses, indicating how they handled the corresponding request. This field may simply include a cache hit versus miss flag, or more detailed debugging information.

RFC 9211 aims to standardize how caches communicate this information, but sadly, it has not yet been widely adopted [18]. Instead, each web cache technology uses its own custom headers to this end, and these are not always officially documented. Mirheidari et al., however, compiled a list of the different header names and values observed when interacting with a selection of popular stand-alone cache servers and CDNs [17].

## 2.5 Cache Pollution

Achieving good cached data locality through optimizing access patterns, adopting the appropriate cache eviction strategies, and designing cache hierarchies have been studied in computer science for decades. There is an immense body of literature on this topic as it applies to software, hardware, and network engineering domains. These works define the term *cache pollution* to broadly refer to conditions that lead to a disruption of the cached data locality, resulting in a decrease in cache hits, and therefore performance degradation. Cache pollution is not necessarily an adversarial event, but rather, an outcome of less-than-optimal cache design given the use case. As an early but representative example relevant for a web application context, researchers showed that automated access patterns of web robots significantly decreased cache locality for popular objects [2].

Cache pollution can also be induced for malicious purposes, aptly called a *cache pollution attack*. Core attack concepts

generally apply to all caches, and the adversarial techniques presented in all works can be summarized in two categories: 1) Attacks that repeatedly request the same unpopular objects, and 2) attacks that request a wide array of objects. Both techniques hurt the locality of genuinely popular objects. Defenses fall into various anomaly detection approaches based on access characteristics and patterns, and specialized cache replacement algorithms to minimize attacker influence [25].

For network applications of caches, the vast majority of literature is focused on Information-Centric Networking (ICN), which encompasses Content-Centric Networking (CCN) and Named Data Networking (NDN). ICN is a novel network communications paradigm for the Internet that revolves around requesting named content, instead of host-to-host packet exchanges [26]. In-network caching plays a prominent role in this architecture, which resulted in a plethora of scholarly works exploring pollution attacks and defenses for the ICN paradigm (e.g., [6, 11–15, 19, 27, 29, 30]). We omit the specifics of individual works, as ICN is not relevant to our paper’s scope. However, the attack and defense contributions are in line with the general summary we provided above.

Gao et al. and Deng et al. present incremental works more closely related to ours, exploring cache pollution on the web [7, 9]. These discuss the same two attack categories as applied to caching proxies and DNS caches, propose an anomaly detection scheme based on statistics computed over request features, and perform experiments using Squid. They primarily focus on pollution attacks in web and peer-to-peer cache deployments using forward proxies, as opposed to the *reverse* proxies more prevalent today that we focus on. Nonetheless, the authors argue (and we agree) that the findings are also applicable to reverse proxies. More recently, Afek et al. explore a similar attack against DNS caches [1]. While the idea of flushing a cache via adversarial input is shared, the operational characteristics, defenses, and implications differ significantly between DNS and web caches.

In the next section, we formulate WCO, our novel approach to cache pollution. We defer discussion of the key differentiators between our work and the above literature until then.

### 3 Research Statement

#### 3.1 The Problem

The observation that motivates our research is that, while cache busting is often seen as a harmless and sometimes useful property of cache-enabled web architectures, the factors that enable cache busting can be used to generate redundant cache entries and increase eviction pressure.

We elaborate on the details of this observation and our resulting hypothesis below.

Let:

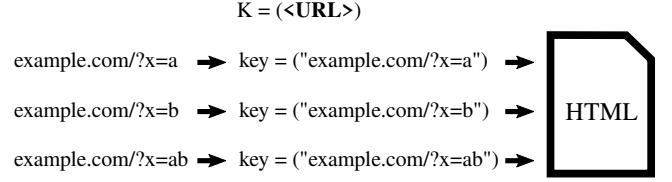


Figure 1: Cache pollution due to imprecise cache key definition, keyed on the full URL.

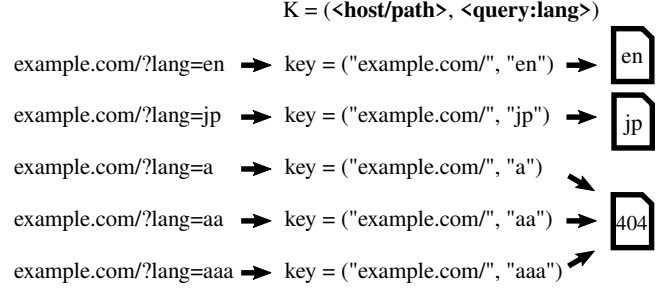


Figure 2: Cache pollution due to imprecise cache key definition, keyed on the query string parameter “lang”, combined with insufficient validation of its value.

- $K = (k_1, \dots, k_n)$  be the  $n$ -tuple that represents the cache key definition, where  $k_i \in K, i = 1, \dots, n$  describe the keyed HTTP request elements.
- $request_i, i \in \mathbb{N}$  be *distinct* HTTP requests that result in cacheable responses containing payloads  $object_i$ .
- $key_i$  be the concrete cache keys derived from  $request_i$  according to the definition  $K$ , resulting in the cache store mapping  $key_i \rightarrow object_i$ .

Then:

For any two  $key_i \rightarrow object_i$  and  $key_j \rightarrow object_j$ ,

if  $key_i \neq key_j$ , but  $object_i = object_j$ ,

we say that  $K$  is *imprecise*.

Intuitively, a cache key definition that includes keyed request elements that do not influence the response payload results in the identical object getting stored multiple times, but under different cache keys, wasting space. This is a generalization and formalization of the basis of all cache busting, which may then be systematically abused for cache pollution. Web clients are free to craft and send any HTTP request to a server, and therefore, they can manipulate any keyed element included in  $K$  in an attempt to achieve this effect.

The semantics of a *precise* cache key definition naturally follow: Every unique concrete cache key derived from a

precise definition maps to a unique cached object, eliminating this cache pollution vector.

Note that an imprecise cache key definition is a prerequisite, but not a sufficient condition for effective abuse. Whether cache pollution via imprecise cache key definitions is viable depends on request validation, which responses are cacheable, cache capacity, object-size distribution, and traffic rates.

Figure 1 depicts an example where the imprecise cache key definition leaves little room for mitigating abuse through validation. Here, the cache key is defined as the entire URL, including the host, path, and the complete query string. As a result, an attacker can attach arbitrary query strings that are not recognized by the application to repeatedly cache the homepage under different cache keys.

Figure 2, on the other hand, illustrates a more complex situation. The cache key is now defined more prescriptively to only include the host, path, and a specific query string parameter that the application uses to serve a homepage translated into different languages. This mitigates the attack in the previous example, since arbitrary query strings do not influence the cache key. However, there is still room for abuse: If 1) the application does not validate the parameter against a list of supported languages, but allows any arbitrary string, and 2) the cache rules allow caching of 404 error pages—a common practice for traffic optimization—, then the attacker can still pollute the cache by repeatedly caching the same error page under different values for “lang”.

Given the above observations, we present our hypothesis: An arbitrary Internet attacker can abuse imprecisely defined cache keys, keyed on insufficiently validated HTTP elements, to create redundant cache entries, launching pollution attacks. Sustained generation of such entries reduces cache effectiveness and may increase origin load or facilitate eviction-dependent attacks. We call this attack Web Cache Overflow (WCO).

## 3.2 Impact and Novelty

WCO fills a web cache with redundant copies of a specific object, forcefully purging everything else in the process. This negates the benefits of using a cache, drastically increasing the traffic load on the origin server and round-trip times for clients interacting with the application, ultimately resulting in a DoS similar to other cache pollution attacks.

However, WCO has a significant property that differentiates it from previous work: it requires no knowledge of the victim cache’s regular access patterns, the popularity of objects served from the target website, or the cache eviction algorithm in use. Existing attacks rely on measurements and accurate estimations of such details for calculated, complex strategies that selectively boost the locality of unpopular files or disrupt the locality of popular ones, whereas such information is immaterial for WCO. The attack methodology requires no prior knowledge of the target environment and

instead results in indiscriminate purging of objects, enabling even unsophisticated attackers. The sole requirement, identifying imprecise cache keys, can be trivially automated, as we discuss in Section 4 and later demonstrate in Section 5.1. Moreover, our work presents a fundamentally different attack vector and threat model compared to prior studies: previous attacks rely on pre-existing unpopular objects to fill the cache, inherently limiting their scale to the number and size of such objects available on the target site, whereas our approach employs cache-busting techniques to actively create arbitrary cache entries, removing this constraint. Thus, our attack is limited not by the site’s content but only by factors such as available bandwidth, server-side rate limiting, the expiration age of cached entries, and the cache capacity. Finally, our methodology avoids the high cost of downloading large files by leveraging HEAD requests, further reducing the overhead of launching the attack.

Furthermore, WCO’s ability to purge objects facilitates other attacks such as cache poisoning and cache-based side channels, which usually come with a pre-condition attached: The resource targeted with these attacks must not already be cached. Assume an attacker discovers a reflected cross-site scripting (XSS) vulnerability, where the application includes parts of the attacker-controlled request on the rendered page without input validation or output sanitization, allowing the attacker to inject a malicious JavaScript snippet into the page. If this poisonable page is also cacheable, an attacker may attempt to store the response containing the reflected XSS payload. While such vulnerabilities are common, exploiting them is challenging if the page in question is getting a lot of traffic, implying that the original copy will perpetually be cached, not allowing the attacker to override it with a poisoned version. While WCO does not deterministically select the entry evicted by the replacement policy, sustained eviction pressure may create a window of opportunity for the target to become absent and for the attacker to poison the cache.

## 3.3 Research Questions

Our overarching goal is to test our hypothesis and characterize the conditions under which WCO degrades cache effectiveness by addressing the following research questions.

- Q1 Are imprecise cache key definitions a common occurrence in real-life websites?
- Q2 Under which deployment conditions is WCO practical? How does it affect cache effectiveness, and what traffic cost is required to maintain it?
- Q3 How do various cache configurations, such as the total cache capacity, cached object size, web cache technology used, etc., influence the attack?

Q4 Can WCO be mitigated? How effective are the established cache defenses against WCO?

### 3.4 Threat Model

We adopt a standard web application threat model. The attacker is an arbitrary Internet user, in full control of their user agent, with the capability to craft HTTP requests to a web application fronted by a server-side cache. The attacker cannot control cache admission or replacement and cannot select which legitimate entry is evicted. Whether the transport is secure is irrelevant to our work.

## 4 Methodology

---

**Algorithm 1** Pseudo-code for the high-level WCO flow.

---

```

Input target: Victim cache-fronted website.
1: path, keySpec  $\leftarrow$  findObjectWithImpreciseKey(target)
2:
3: # Procedure 1: Fill the cache with redundant objects
4: busterList  $\leftarrow$  []
5: repeat
6:   busterRequest  $\leftarrow$  generateCacheBuster(path, keySpec)
7:   sendGET(busterRequest)
8:   busterList.append(busterRequest)
9: until isCacheFull() = True
10:
11: # Procedure 2: Refresh the TTL of cached objects
12: loop
13:   for all busterRequest  $\in$  busterList do
14:     response  $\leftarrow$  sendHEAD(busterRequest)
15:     if response.cacheStatus = Miss then
16:       sendGET(busterRequest)
17:     end if
18:   end for
19: end loop

```

---

We now describe how to achieve WCO in concrete steps. Conceptually, WCO is straightforward, but there are important considerations and optimization opportunities hidden in the details. We first provide a simplified view into the core methodology for brevity, and later discuss those considerations.

### 4.1 Overview

WCO follows the steps below, also illustrated in Algorithm 1 with specific lines referenced in the text.

1. Pick a target victim website fronted by a web cache. Prior knowledge of the cache capacity, cache eviction algorithm, or traffic patterns to the website is **not** necessary. However, if this information is available, it can be used to optimize the attack.
2. Identify a cacheable object specified by an imprecise cache key (line 1). This is the object that the attack will fill the cache with, and large files are better.

3. Abuse the imprecise cache key definition to generate a unique cache buster; i.e., a request for the cacheable object with the appropriate keyed element modified to cause a cache miss, resulting in a fresh copy of the object being fetched from the origin and stored under a new cache key. Repeat this process until the cache is approximately filled to capacity with redundant objects. Also, keep a list of all generated cache busters (lines 3-9).

At this stage, the origin is suffering the full consequences of the attack due to cache degradation, resulting in repeated cache misses for legitimate traffic. The number of entries in the list of cache busters we maintain matches the number of objects needed to fill the cache.

4. To maintain the attack and prevent legitimate traffic from eventually recovering the cache locality for popular objects, cycle through the list of cache busters used for filling the cache in the previous step, and probe them with HEAD requests (lines 11-19). The HEAD request refreshes the cache time-to-live (TTL) of the bogus object, preventing it from getting evicted.
5. If the response for any HEAD request indicates a cache miss, that implies that the corresponding bogus object was already evicted. Therefore, restore it to the cache by sending a new GET request with the same cache buster (lines 15-16).

In Algorithm 1, the routines *sendGET* and *sendHEAD* are self-explanatory. We discuss the undefined routines *findObjectWithImpreciseKey* and *isCacheFull* later in the following subsections.

### 4.2 Cost Considerations

As with any volumetric attack, WCO is only meaningful when the damage inflicted on the origin server outweighs the cost incurred by the attacker. We define the metric *Traffic Amplification Ratio (TAR)* to measure the effectiveness of the attack based on the disruption of the cache hit rate. TAR is calculated by measuring the increased traffic that reaches the origin server due to cache misses resulting from the attack, divided by the attacker's bandwidth expense. A higher value indicates greater disruption relative to the attacker's resource investment.

$$\text{TAR} = \frac{T_{\text{attack}} - T_{\text{normal}}}{T_{\text{WCO}}}$$

where:

- $T_{\text{normal}}$ : Volume of traffic reaching the origin server due to cache misses under normal conditions.
- $T_{\text{attack}}$ : Volume of traffic reaching the origin server due to cache misses during the attack.

- $T_{WCO}$ : Traffic generated by the attacker.

The target cache’s capacity and the size of the redundantly stored object directly factor into the cost, as they influence the number of requests necessary to fill the cache and maintain the attack. Consequently, the use of HEAD requests during attack maintenance is intentional. Since responses for HEAD requests do not contain the body payload, this greatly reduces client-side bandwidth and the attack cost. TAR measures attacker efficiency rather than absolute service degradation. Consequently, a configuration can retain a comparatively higher hit rate yet produce a larger TAR when maintaining the attack requires substantially less attacker traffic.

Beyond these general considerations, cache implementation behavior may further reduce client-side bandwidth. We identified two such cases in our experiments.

First, servers may be configured to perform opportunistic caching by upgrading a client’s HEAD requests to GETs before forwarding them to the origin and storing the resulting object without transferring it back to the client. All GET requests in our methodology can be replaced with HEADs and lines 15-17 in Algorithm 1 eliminated, resulting in the same disruption, but without making the attacker incur the cost of receiving the response payload. We observed this behavior with the default configurations of NGINX and Varnish.

Second, servers may allow for early termination of connections while still caching the response, allowing the attacker to issue GET requests but not read the response, resulting in a similar bandwidth optimization. In our experiments, Squid was the sole server that required the client to consume the entire body before caching an object, while all other caches allowed early termination.

Also note that while the legitimate traffic received at the website will compete with the attacker’s probes for cache space and result in some bogus objects getting evicted until they can be restored via the TTL refresh loop (Steps 4–5 in the overview, lines 11–19 in Algorithm 1), that does not imply better traffic offload for the origin. Since the cache is still at storage capacity, even this temporary caching of genuinely popular content will result in cache thrashing and fail to remediate the cache degradation. We demonstrate this effect later in our evaluation, in Section 5.

### 4.3 Finding Objects With Imprecise Cache Keys

Identifying a cacheable object with an imprecise cache key definition, ideally a large one, hosted on the target website, is an essential part of WCO. Thankfully, this process can be automated. Our approach for identifying such exploitable objects leverages the methodology previously presented by Golinelli et al. in their work that describes how to detect hidden web caches, and extends it for our needs [10].

Specifically, we implement a recursive web crawler that is seeded by the target website’s domain. The crawler issues GET requests for the resources linked from the crawled pages, and checks the response for indications of caching. These include response header heuristics compiled from the work of Mirheidari et al. [17], supplemented by us to include the headers introduced in RFC 9211 [18].

Once the crawl is complete, yielding a list of cacheable objects, we sort these by object size in descending order, and work through the list to automatically reverse engineer their cache key definitions. This process involves crafting a GET request for that object, systematically modifying the frequently keyed HTTP elements in isolation, sending the mutated request, and checking the response for cache status indicators using the aforementioned header heuristics. In particular, we test the following, in the given order:

1. Modify the query string.
2. Include and modify the headers drawn from keyed fields observed in default configurations of popular cache technologies (i.e., Origin, User-Agent, X-Forwarded-Host, X-Method-Override, X-Forwarded-Scheme, Accept-Language, Accept-Encoding, Accept, X-Forwarded-For).
3. Edit the values of headers specified with the Vary header.
4. Include `Cache-Control: no-cache`.
5. Add arbitrary cookies.

For each mutation, we compare the response representation with the baseline and inspect the cache-status indicators. We stop after finding one object for which an equivalent response can be stored under a distinct key because this establishes the prerequisite condition for the attack.

This is depicted on line 1 of Algorithm 1 with the routine *findObjectWithImpreciseKey*, which returns both the path for the exploitable object and the imprecise cache key definition, allowing us to generate an arbitrary number of cache busters by varying the appropriate keyed HTTP element.

### 4.4 Detecting When the Cache Is Full

Finally, an approximation of when the cache is full and the attack is at its peak is an important signal for the attacker. On one hand, over-approximating the number of redundant objects to cache is costly, as generating new cache busters and sending GET requests for them incur a higher cost than the TTL maintenance loop of sending HEAD requests, providing no real benefit in the process. On the other hand, under-approximating this number means that the attacker switches to the maintenance loop too early, leaving space in the cache for genuinely popular objects, and hence under-utilizing WCO. We propose two methods to tackle this problem.

First, if the cache capacity can be estimated, dividing it by the known stored object size approximates the number of entries required to occupy that capacity. This information may not be known for proprietary deployments. However, note that many production systems rely on pre-packaged software with baked-in defaults (e.g., caching proxies deployed via containers in public registries, managed cache components provided by hyperscalers like AWS, Azure, GCP), making informed estimations practical if contextual clues for the deployment setup exist. We use a similar methodology in our evaluation, in Section 5, to collect information on commonly seen cache sizes.

Second, when cache capacity estimation is not feasible, attackers can monitor cache hit rate. Specifically, one can select a highly popular cacheable object (e.g., an image banner on the homepage), periodically issue requests for that object while the attack is in progress, and monitor the cache hits and misses. Observing a cache miss for an object served from the homepage is unlikely and could alone indicate that the attack has escalated to high severity. The attacker can then decide whether to cache more objects or switch to TTL maintenance depending on the cache hit/miss rate calculated over these monitoring probes.

## 5 Evaluation

We now evaluate how WCO performs in practice through a set of detailed experiments, and start answering the research questions we laid out in Section 3.

Testing WCO on production websites is not ethically feasible. DoS attacks are explicitly prohibited on major bug bounty platforms, and even throttled volumetric tests can impose performance and cost consequences on website operators.

We therefore evaluate the attack in a controlled laboratory environment, attacking our own infrastructure. This is aligned with all the cache pollution works we cited in Section 2, and is the overall standard approach in DoS research. To ensure our evaluation methodology is realistic, we first perform non-disruptive Internet measurements and use the observed object sizes and configured cache capacities to select parameters for our controlled experiments.

### 5.1 Imprecise Cache Keys and Object Size In The Wild

Identifying cacheable objects with imprecise cache keys is the prerequisite for WCO. Therefore, we first perform a large-scale Internet experiment to determine whether imprecise cache keys indeed exist in popular websites. While doing so, we also measure the sizes of these cache-bustable objects, as this is a critical parameter that influences the viability of WCO that we must evaluate later.

We conduct this measurement on the Tranco top 10k domain list generated on April 7, 2025 [20].<sup>1</sup> We follow the methodology in Section 4.3: we crawl each website, identify cacheable objects, and automatically reverse engineer their cache keys. We limit the crawl to a maximum of 10 pages on each of at most 10 unique subdomains per website to avoid generating unreasonable traffic volumes. During cache-key inference, we process objects by size in descending order and stop when we find the largest object with an imprecise cache key. Our objective is not to perform a comprehensive measurement study detailing all exploitable objects, but to demonstrate the existence of at least one such object on a website, therefore minimizing the experiment’s burden on production systems.

Out of the 10k websites in our dataset, we were able to crawl 5437, while the rest did not respond to our HTTP requests because of infrastructure issues or possible bot-detection defenses. Among the websites successfully tested, 4000 contained cacheable objects, and **3600** of these contained at least one object with an imprecise cache key. We did not detect the condition for any tested object on the remaining 400 websites. The five-number summary for the affected objects is as follows:  $Min = 2\text{bytes}$ ,  $Q1 = 0.12\text{MB}$ ,  $Median = 0.37\text{MB}$ ,  $Q3 = 1.02\text{MB}$ ,  $Max = 187.54\text{MB}$ .

We note that these numbers are lower bounds due to the limits we imposed on the crawl for ethical considerations (e.g., we did not make attempts to bypass bot detection techniques, which are commonly utilized for large file downloads). Case in point, our experiment did not catch a 6 GB operating system image hosted on one of the tested sites, which we later manually confirmed to have an imprecise cache key.

To empirically assess this lower bound, we randomly sampled 100 websites from the subset with affected objects smaller than the median size and manually searched each site for larger objects exhibiting the same condition. 16 sites redirected to other domains and were discarded. On 72 sites, we found larger affected objects; on 12 sites, we did not. The five-number summary of size ratios between the largest manually discovered affected object and the automatically detected one is:  $Min = 2.96$ ,  $Q1 = 43.84$ ,  $Median = 148.53$ ,  $Q3 = 1605.43$ ,  $Max = 59507.35$ . This indicates that in many cases, significantly larger cacheable objects with imprecise cache keys exist on the same website.

This confirms that imprecise cache keys are indeed very common, and they can be automatically detected, answering our first research question affirmatively (i.e., Section 3.3, Q1). As for the object size distribution, we confirm that more than 25% of websites have exploitable objects larger than 1 MB, extending into the 100 MB range. However, the median is fairly small at 0.37 MB. We leverage the insights from this measurement in the rest of our evaluation, and test how these object sizes influence the success of WCO.

<sup>1</sup> Available at <https://tranco-list.eu/list/LJL44>.

## 5.2 Cache Capacity In The Wild

To establish realistic cache capacity values to use in our experiments, we selected five popular open-source caching proxy technologies (i.e., Apache Traffic Server (ATS), HAProxy, NGINX, Squid, and Varnish), and searched GitHub for projects that use them via the GitHub API. We then analyzed the cache configurations these projects are bundled with.

This process yielded 127 projects with valid cache configurations; a sample list is available in Appendix B. Table 1 summarizes the results, showing that the median cache capacity does not exceed 1 GB for any proxy in this dataset. Most observed configurations were below 50 GB, with a small number of ATS outliers exceeding 100 GB. We designed the rest of the experiments in this section in light of these findings.

## 5.3 WCO With Common Defaults

We perform our first WCO experiment with the same five caching proxies. This experiment tests whether the redundant-entry mechanism produces cache degradation across implementations under one uniform configuration informed by the preceding measurements.

We configure all proxies to cache every response. We work with objects of size 1 MB, and we set the cache capacity to 50 GB, both informed by our observations in the previous experiment. The only exception is HAProxy, for which the maximum cache capacity is 4095 MB. Other configuration parameters are left at their default settings. In particular, all caches employ the Least Recently Used (LRU) eviction algorithm, and they define their cache keys to include the query string, allowing us to perform cache busting by modifying its value.

To simulate the routine traffic a website receives, we use Web Polygraph, a benchmarking tool for caching proxies and other web intermediaries [21]. Web Polygraph provides a client component that generates traffic and a server component that responds to it. We place the tested cache between them and measure the cache hit rate.

Web Polygraph provides predefined workloads simulating common traffic patterns. We use the workload `simple.pg`.<sup>2</sup> This workload represents a mix of repeated and non-repeated requests with a configurable request rate, which directly affects the cache hit rate during the experiment. By default, the recurrence rate is set to 55%, meaning that Web Polygraph requests the same resources 55% of the time. Since we set all responses to be cacheable, we anticipate a consistent hit ratio of around 55%. We configure Web Polygraph to issue 100 requests per second.

We test each cache in isolation by deploying it as a reverse proxy in our lab environment, in front of our origin server that serves static files of varying sizes. The attacker’s requests

<sup>2</sup><https://github.com/albertok/web-polygraph/blob/master/workloads/simple.pg>

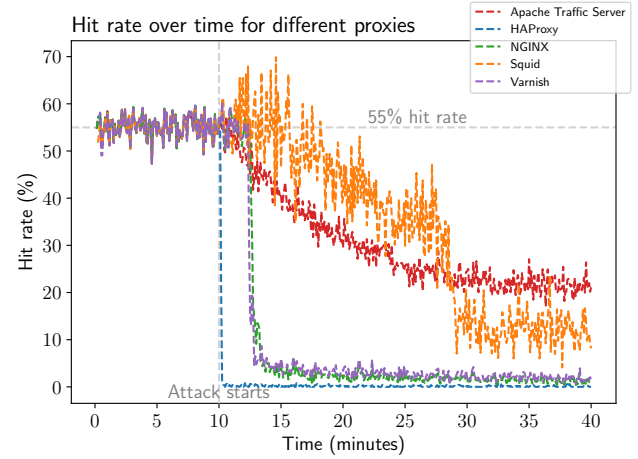


Figure 3: Cache hit rate before and during WCO with different caching technologies. The cache size is 50 GB, while the object size is 1 MB.

are routed to this origin, while the simulated routine traffic is directed to Web Polygraph’s server. The two routes share the same cache. As a result, the impact of WCO manifests in Web Polygraph’s benchmarks.

For each experiment, we first run Web Polygraph for 10 minutes without performing any attack to warm the cache. We then run WCO for the next 30 minutes. During both phases, we record the cache hit rate. Compared to the subsequent experiments, we use a longer attack duration here to clearly illustrate the attack’s warm-up and maintenance phases.

Table 2 presents the average cache hit rate for each proxy before and during WCO. The results show that WCO significantly degrades performance in all cases. The cache hit rate drops further with the increase in the origin server’s traffic load. Some caches retain a somewhat higher residual hit rate under our setup, but we do not interpret this as inherent resilience. Rather, these differences likely reflect implementation-specific behavior under the single attack configuration we applied uniformly across all caches. A more sophisticated, cache-specific attack strategy could plausibly shift these relative results further. Figure 3 visually depicts the cache hit rate drop as the attack progresses.

The results so far are promising, demonstrating that WCO renders the cache ineffective under widely used default configurations.

## 5.4 WCO With Varying Parameters

We next evaluate how cache capacity and cached object size influence the outcome. Having reproduced the mechanism across five cache technology implementations, we use NGINX for the remaining controlled experiments.

Table 1: Cache capacities configured in the analyzed GitHub projects. The # column gives the number of configurations. Min and Max are the smallest and largest configured values observed in this dataset.

| Proxy   | #  | Min (MB) | Q1 (MB) | Median (MB) | Q3 (MB) | Max (MB) |
|---------|----|----------|---------|-------------|---------|----------|
| ATS     | 23 | 144      | 256     | 256         | 10240   | 1048576  |
| HAProxy | 34 | 4        | 20      | 164         | 512     | 4095     |
| NGINX   | 29 | 73       | 256     | 1024        | 1024    | 51200    |
| Squid   | 21 | 16       | 100     | 512         | 5000    | 51200    |
| Varnish | 20 | 1        | 256     | 256         | 1280    | 32768    |

Table 2: Cache hit rate (HR) before and during WCO. Values are averages over the 10-minute phases.

| Cache   | Before HR (%) | During HR (%) |
|---------|---------------|---------------|
| ATS     | 55.1          | 30.0          |
| HAProxy | 55.1          | 0.5           |
| NGINX   | 55.1          | 6.9           |
| Squid   | 55.0          | 32.9          |
| Varnish | 55.0          | 7.2           |

Table 3: WCO cost with an object size of 1 MB and varying cache capacity.

| Cache Capacity (GB) | $T_{\text{normal}}$ (MB) | $T_{\text{attack}}$ (MB) | $T_{\text{WCO}}$ (MB) | TAR   |
|---------------------|--------------------------|--------------------------|-----------------------|-------|
| 100                 | 351.37                   | 678.25                   | 153.77                | 2.13  |
| 50                  | 348.42                   | 684.02                   | 133.55                | 2.51  |
| 10                  | 349.98                   | 707.79                   | 36.05                 | 9.92  |
| 1                   | 348.76                   | 618.35                   | 3.64                  | 74.05 |

In these experiments, we also measure bandwidth and calculate the Traffic Amplification Ratio (TAR) as the parameters vary, to realistically validate that WCO is a practical DoS attack. During maintenance, we throttle the attack traffic so that every redundant entry is probed approximately once every 60 seconds, rather than sending requests as quickly as possible. This interval provides a consistent basis for comparing attacker efficiency; different TTLs, admission policies, and legitimate traffic would change the maintenance rate required in a deployment.

#### 5.4.1 Cache Capacity

We run four tests with NGINX configured with a cache storage capacity ranging from 1 GB to 100 GB, informed by our previous measurement experiment. We also repeated the attack using several objects of different sizes in our exploratory studies, ranging from 100 KB to 500 MB; here, we present the results with a 1 MB file.

Figure 4 shows the cache hit rate before and during the attack, once again demonstrating that WCO rapidly degrades

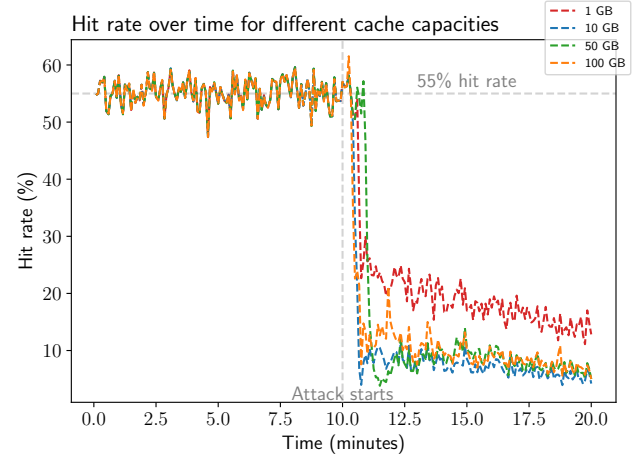


Figure 4: Cache hit rate before and during WCO with NGINX for various cache capacities and an object of 1 MB.

performance in the same way, regardless of the cache capacity. However, we also see in Table 3 that the cache capacity has a notable impact on the attacker’s cost. Larger caches require more objects to fill, which in turn translates to higher bandwidth. We stress that this is a conservative case. As we show in the next experiment, the attacker could employ larger objects to perform the attack efficiently on a 100 GB cache. Recall that the attacker does not need to download the full body of each response, as explained in Section 4, which is why the bandwidth values are significantly lower than the cache capacity.

#### 5.4.2 Object Size

We now configure NGINX with a cache capacity of 50 GB and perform WCO using four different object sizes, ranging from 100 KB to 500 MB.

Figure 5 shows the resulting cache hit rate, demonstrating that WCO works regardless of object size. Table 4 confirms our intuition that larger objects yield far better TAR values, but even with a 1 MB object, WCO remains viable.

Table 4: WCO cost with a cache capacity of 50 GB and varying object sizes.

| Object Size<br>(MB) | $T_{\text{normal}}$<br>(MB) | $T_{\text{attack}}$<br>(MB) | $T_{\text{WCO}}$<br>(MB) | TAR   |
|---------------------|-----------------------------|-----------------------------|--------------------------|-------|
| 500                 | 357.82                      | 676.78                      | 15.31                    | 20.84 |
| 10                  | 351.92                      | 583.28                      | 7.76                     | 29.83 |
| 1                   | 348.42                      | 684.02                      | 133.55                   | 2.51  |
| 0.1                 | 350.65                      | 571.81                      | 391.43                   | 0.57  |

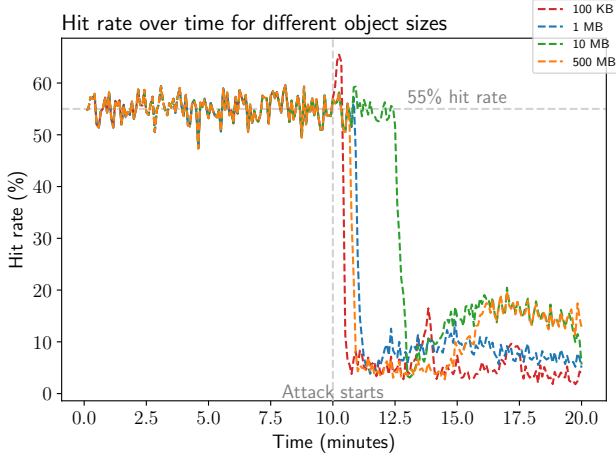


Figure 5: Cache hit rate before and during WCO with NGINX for different object sizes and cache capacity of 50 GB.

## 5.5 Eviction Algorithms

LRU is the most common, and often the only supported eviction algorithm, with many web cache technologies. Yet, some of the works we cited in Section 2 specifically explore the use of other caching strategies in their defenses against cache pollution (e.g., [13]). Therefore, we also evaluate the impact of different eviction algorithms on the effectiveness of WCO.

For this particular experiment, we use Squid instead of NGINX, since only Squid supports the variety of eviction algorithms we would like to test. We set the cache capacity to 50 GB, and the cached object size to 1 MB. We then run tests for each eviction algorithm that Squid supports: LRU, heap LRU, heap GDSF, and heap LFUDA.

Table 5 presents the results of this experiment. In short, we saw no difference of note between these different eviction algorithms, and we conclude that WCO is robust against tweaks to the caching strategy. This confirms our previous claim that cache eviction details are irrelevant for WCO’s abuse mechanism of filling a cache to capacity.

Table 5: WCO impact on Squid with different eviction algorithms.

| Eviction Algorithm | Before HR (%) | During HR (%) |
|--------------------|---------------|---------------|
| Heap LRU           | 55.3          | 36.7          |
| LRU                | 55.3          | 34.2          |
| Heap GDSF          | 55.3          | 35.2          |
| Heap LFUDA         | 55.3          | 35.2          |

## 5.6 WCO-facilitated Cache Poisoning

We next evaluate WCO as an eviction-pressure mechanism that may compose with an independent cache-poisoning vulnerability. This experiment is a proof of composition against a deliberately vulnerable laboratory application.

We add to our setup an origin application containing a reflected XSS vulnerability whose response is cacheable and poisonable. We use a 1 MB object and vary cache capacity. We restrict this experiment to smaller caches because the attack did not succeed within a practical experimental duration for larger configurations. Attackers with more resources or larger objects could overcome this limitation and poison larger caches.

We start the test in a state where a response for the vulnerable endpoint is already cached, as it would be in a real-world scenario. The vulnerable endpoint regularly receives requests from simulated legitimate clients. The attacker generates redundant entries while repeatedly attempting to exercise the independent poisoning vulnerability. The attacker cannot select which object is evicted. If the target becomes absent, the attacker has an opportunity to store the malicious response; a legitimate request may instead reinsert the benign response, requiring the attacker to maintain eviction pressure and wait for another opportunity.

We let this scenario play out with simulated traffic representing arbitrary clients. The attacker performs WCO as usual, while simultaneously trying to poison the vulnerable endpoint, competing with regular traffic. We experiment with various routine request rates for the endpoint, and we perform 5 trials for each of these experiments to measure the average time for a successful poisoning attack.

Table 6 shows that success depends on both cache capacity and legitimate request rate. Several 4 GB configurations did not succeed within the 15-minute timeout, and the mechanism did not select the target for eviction. In this deliberately vulnerable laboratory application, redundant-entry flooding increased the opportunities for a previously cached endpoint to become absent, facilitating exploitation of previously immaterial vulnerabilities.

Table 6: Average time to successfully perform cache poisoning by leveraging WCO to purge a target object with different cache capacities and varying regular traffic rates. “-” means the attack did not succeed within a 15-minute timeout.

| Request Rate (r/s) | Average Time to Success (s) |         |         |
|--------------------|-----------------------------|---------|---------|
|                    | 1024 MB                     | 2048 MB | 4096 MB |
| 1                  | 32.0                        | 49.0    | 15.8    |
| 10                 | 30.2                        | 50.0    | -       |
| 50                 | 18.0                        | 63.8    | -       |
| 100                | 92.6                        | 108.4   | -       |

## 5.7 Summary of Results and Limitations

Our measurements over Tranco top 10k show that more than one third of these popular websites have cacheable objects with imprecise cache keys, confirming the premise for WCO.

The remaining experiments demonstrate that the core mechanism of WCO, that is, filling caches with duplicate objects, forcing the eviction of everything else, and leaving the cache in a perpetual state of cache thrashing, is viable with all tested caching proxies, regardless of the parameters such as cache capacity and exploitable object size. These parameters, however, do have practical impacts on whether WCO is viable as a DoS vector. Extreme values (very small files, very large caches) can become cost-prohibitive, resulting in low TAR values. This is an inherent limitation of the mechanism and methodology we propose in this research, and there are cases where WCO may not be an effective attack. However, as also evidenced by our study of commonly seen object sizes and cache capacity configurations in the wild, such extreme values for these parameters are not the norm, and there are many opportunities to launch viable WCO attacks in realistic web application deployment settings.

All in all, we conclude that the findings we presented here successfully address our research questions Q1, Q2, and Q3 from Section 3.3, establishing WCO as a practical threat.

## 6 Mitigations

To answer our last research question Q4, we describe several mitigation strategies for WCO and test their effectiveness.

### 6.1 Cache Deduplication

An effective mitigation against WCO is deduplication. Cache deduplication may be implemented by computing checksums of cacheable responses as they are received, comparing them against the checksums of already cached objects, and only storing a new copy if the object is unique in storage. Otherwise, a data structure would be updated to associate the distinct cache keys with a single stored copy of the object.

Table 7: Average CPU time over 1000 runs to compute file hashes and checksums.

| File Size | xxHash            | SHA-512           |
|-----------|-------------------|-------------------|
|           | Avg. CPU Time (s) | Avg. CPU Time (s) |
| 500 MB    | 0.24258           | 0.81138           |
| 10 MB     | 0.00548           | 0.02170           |
| 5 MB      | 0.00288           | 0.01296           |
| 1 MB      | 0.0007            | 0.00304           |
| 500 KB    | 0.0004            | 0.00161           |
| 100 KB    | 0.00013           | 0.00042           |

While deduplication is a well-known technique designed for this exact problem, its application to web caches could be challenging. Both the checksum computation and the cache key tracking data structure management tasks can become prohibitively costly for a busy server, and worse, cause processing delays, increasing application response times.

We run a simple experiment to estimate this cost by checksumming files of various sizes and measuring the time the operation takes. We test a cryptographic hash (i.e., SHA-512), and also a non-cryptographic checksum (xxHash, specifically the XXH3 variant [28]) designed to be very fast to compare. We perform 1000 trials for every experiment, and present the average times elapsed.

Table 7 summarizes the results. This may appear negligible in isolation, but given that today’s production platforms process millions of requests per second (e.g., [3]), processing cycles and latency may quickly become unreasonable.

### 6.2 Anomaly Detection

Anomaly detection is the main defense approach adopted by nearly all cache pollution defense works we referenced in Section 2. Most relevant for our context, the works by Gao et al. and Deng et al. that present cache pollution attacks and defenses on forward web caches propose a detection scheme based on various request and traffic features [7, 9]. These works show that their approach is effective, but it also comes with processing overhead and false positives. Moreover, it can take up to 10 hours to catch anomalies. These downsides are not unique to the cited works and are expected of any anomaly detection approach to security.

WCO can also be detected in this manner, likely more easily than other cache pollution attacks, due to its noisier nature. If the shortcomings of anomaly detection are acceptable, this could be a viable solution. However, we also expect the costs to be material for a real-life application, and as discussed in the cited works, IP blocking capabilities themselves may be weaponized by an attacker to block NAT gateways. All in all, we do not believe anomaly detection should be the preferred approach if a more effective solution can be found.

Table 8: WCO hit rates with different rate limits, a cache of 50 GB and an object of 1 MB.

| Rate Limit (req/s) | Before HR (%) | During HR (%) |
|--------------------|---------------|---------------|
| 50                 | 55.1          | 54.9          |
| 10                 | 54.6          | 54.6          |
| 5                  | 38.4          | 37.8          |
| 1                  | 11.7          | 10.1          |

### 6.3 Rate Limits

WCO generates a large volume of sustained traffic over a long time for the TTL maintenance loop. Therefore, we explore rate limiting separately from more advanced anomaly detection as a less costly option. We conduct an experiment with the same evaluation setup to test this approach. We use NGINX, set the cache capacity to 50 GB, and employ a 1 MB file as the cached object. We configure Web Polygraph to issue 10 requests per second from 10 different IP addresses, for a total of 100 requests per second. We run Web Polygraph for 10 minutes without performing any attack, and then run it together with WCO for another 10 minutes. In both phases, we measure the cache hit rate. We repeat this experiment while applying rate limits ranging from 1 to 50 requests per second, calibrating attack intensity to each limit. If the attacker issues requests faster than the configured limit, the rate limiter will begin dropping excess requests. Those drops will discard many of the attacker’s requests and prevent the sustained traffic required to fill the cache. Conversely, an attacker that reduces its rate to be at or below the measured limit can avoid being dropped and, although the attack proceeds more slowly, given sufficient time, it can still gradually populate the cache. In our experiments, we calibrate the attack rate to the applied limit. In the real world, an attacker could easily spot and measure the rate limit and adjust accordingly.

Table 8 shows the cache hit rate before and during the attack. The results indicate that rate limiting is effective in reducing the attack’s impact. That said, rate limits below 10 requests per second, while being effective in reducing the impact of the attack, naturally also impact regular traffic, causing a drop in the cache hit rate before the attack begins. This once again demonstrates that rate limits cannot be applied generally to all traffic without hurting normal traffic flows, and just like anomaly detection, false positives can be disruptive, or they can even be weaponized by an attacker to get legitimate clients blocked.

### 6.4 Stricter Cache-Key Design

Abuse of imprecise cache keys is a necessary precondition for WCO. That also makes eliminating imprecise cache keys the optimal mitigation. In light of all the conceptualization, discussion, and experiments we presented in this work, we

argue that this is the preferred approach.

The main drawback of this approach is that it requires website operators to thoroughly analyze their cache configurations and apply the necessary corrections. At the same time, the same imprecise cache detection methodology we presented in Section 4.3 can be repurposed to run deep crawls on websites and identify all problem cache keys, automating the bulk of the work. Revising the cache key still requires manual work informed by contextual knowledge of the surrounding application, but we argue that this cost is preferable over the challenges of sifting through false positives or suffering a sustained performance overhead.

Case in point, we developed a penetration testing tool that automates the systematic detection of imprecise cache key configurations on websites. The tool functions as a web crawler that recursively explores a target site, collecting and testing all accessible resources. For each resource, it issues a series of HTTP requests, each differing from the baseline request by a random modification to one element; specifically, the query string, selected headers, or cookies. The tool then applies response header heuristics derived from the work of Mirheidari et al. [17] to determine whether a given modification caused the request to bypass the cache. If a modified request results in a cache miss while the corresponding response remains identical to the unmodified version, the tool infers that the cache key configuration is imprecise. The tool provides recommendations to the user on how to patch its findings using natural language. The tool is based on the methodology discussed in this paper and released as open source for researchers and practitioners to use freely in their own environments.

All in all, we assert that the consequences of imprecise cache keys should be communicated to website operators and cache administrators, and crafting strictly precise cache key definitions should be considered a best practice.

## 7 Discussion

### 7.1 Novelty

We now reiterate the differentiating factors between WCO and prior cache pollution strategies. Traditional attacks rely on estimates or knowledge of cache access patterns, object popularity distributions, or replacement behavior. Our results show that WCO operates without such information, as it is solely based on identifying imprecise cache keys: a process that, as demonstrated in Section 4 and Section 5.1, can be trivially automated. This simplicity makes the attack both easier to deploy and more general in scope, removing the need for any target-specific reconnaissance or fine-tuned strategies. WCO actively generates arbitrary cache entries using cache-busting techniques, rather than relying on pre-existing unpopular resources, removing any constraints imposed by the amount and size of the existing content available on the victim site.

WCO is in fact only limited by environmental factors, such as available bandwidth, server-side rate limiting, cache capacity, and the cache expiration policy.

Furthermore, leveraging lightweight HEAD requests instead of full object downloads enables sustained disruption with minimal effort and very low bandwidth and computational cost, making it accessible even to resource-limited adversaries. Finally, the consequences of WCO extend beyond mere cache performance degradation. Its ability to force eviction of cached objects enables new avenues for exploitation, such as cache poisoning or cache-based side channels, by removing the usual precondition that the targeted resource must not already be cached. We discussed how, for instance, an attacker can leverage WCO to create a window of opportunity for turning a reflected XSS vulnerability into a stored one through cache poisoning. Overall, we argue that the results presented in this paper highlight that WCO is not a minor variation of existing cache attacks, but a fundamentally different threat model.

## 7.2 What About Content Delivery Networks?

In this paper, we focused our discussion on server-side caches, and in particular, for our evaluation, stand-alone caching proxies. This scope is not unrealistic; such caching reverse proxies deployed in multiple layers are ubiquitous and essential components of scalable web and cloud architectures. That said, Content Delivery Networks (CDNs) that operate massively distributed Internet overlays of caching reverse proxies also provide immense amounts of caching capacity to the web.

We cannot perform experiments with CDNs due to the aforementioned ethics considerations. Even an ineffective attempt would impose a cost on the infrastructure operators, and volumetric attack testing is explicitly prohibited in the Acceptable Use Policies of all major CDNs we checked.

Even without testing, we assume that WCO would not work on CDNs. While there are no public records of CDN cache sizes, a quick Internet search reveals many speculative accounts of CDNs deploying caches with capacities that are orders of magnitude larger than common stand-alone cache configurations. As evidenced by our evaluations, WCO does not yield a viable TAR value at such extreme values. Therefore, we operate under the assumption that WCO is not relevant for CDNs, and this is a fundamental limitation of our work.

To confirm this assumption and to notify them of our work, we contacted Akamai, AWS CloudFront, CDN77, Cloudflare, Fastly, Google Cloud CDN, KeyCDN, Bunny CDN, and OVHCloud. We shared our findings with them by providing a summary of this research, our proof-of-concept attack tools for their testing, and asking for their feedback.

Akamai, Cloudflare, Fastly, and Google acknowledged the research and reported that the attack would not be feasible at their scale. AWS stated that DoS attacks are outside its bug

bounty scope (even though we did not seek a bounty) and provided no further comment. The remaining parties did not respond to contact attempts.

## 8 Conclusion

We presented WCO, a novel cache pollution attack that abuses imprecise cache keys to fill a web cache with duplicate objects, purging other cached content, and causing cache degradation. Our evaluation confirmed that imprecise cache keys are commonly found on popular websites, and WCO can effectively abuse them in several practical cache deployment setups. Moreover, we showed that WCO’s capability to purge objects on command alone can be a valuable tool in the hands of adversaries. These findings address all four of our research questions, and confirm our hypothesis that WCO is a viable threat.

As we conclude, we reiterate that the crucial lesson learned from this research is that imprecise cache keys can have severe security implications. The resulting issues are best and most efficiently avoided by carefully designing precise cache keys as a general security best practice.

## References

- [1] Yehuda Afek, Anat Bremler-Barr, Shoham Danino, and Yuval Shavitt. A flushing attack on the DNS cache. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 2299–2314, Philadelphia, PA, August 2024. USENIX Association.
- [2] Virgílio Almeida, Daniel Menascé, Rudolf Riedi, Flávia Peligrinelli, Rodrigo Fonseca, and Wagner Meira Jr. Analyzing Web Robots and Their Impact on Caching. In *Workshop on Web Caching and Content Distribution*, 2001.
- [3] Jeff Barr. AWS Identity and Access Management. AWS News Blog, 2021. <https://aws.amazon.com/blogs/aws/happy-10th-birthday-aws-identity-and-access-management/>.
- [4] Cloudflare. Cache Keys, 2025. <https://developers.cloudflare.com/cache/how-to/cache-keys/>.
- [5] Cloudflare. Origin Cache-Control, 2025. <https://developers.cloudflare.com/cache/concepts/cache-control/>.
- [6] Mauro Conti, Paolo Gasti, and Marco Teoli. A Lightweight Mechanism for Detection of Cache Pollution Attacks in Named Data Networking. *Computer Networks*, 57(16):3178–3191, 2013.

- [7] Leiwen Deng, Yan Gao, Yan Chen, and Aleksandar Kuzmanovic. Pollution Attacks and Defenses for Internet Caching Systems. *Computer Networks*, 52(5):935–956, 2008.
- [8] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP Caching. RFC 9111, 2022. <https://datatracker.ietf.org/doc/html/rfc9111>.
- [9] Yan Gao, Leiwen Deng, Aleksandar Kuzmanovic, and Yan Chen. Internet Cache Pollution Attacks and Countermeasures. In *IEEE International Conference on Network Protocols*, 2006.
- [10] Matteo Golinelli and Bruno Crispo. Hidden Web Caches Discovery. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2024.
- [11] Abdelhak Hidouri, Mohamed Hadded, Nasreddine Hajlaoui, Haifa Touati, and Paul Muhlethaler. Cache Pollution Attacks in the NDN Architecture: Impact and Analysis. In *International Conference on Software, Telecommunications and Computer Networks*, 2021.
- [12] Abdelhak Hidouri, Haifa Touati, Mohamed Hadded, Nasreddine Hajlaoui, Paul Muhlethaler, and Samia Bouzefrane. Q-ICAN: A Q-learning Based Cache Pollution Attack Mitigation Approach for Named Data Networking. *Computer Networks*, 235, 2023.
- [13] Amin Karami and Manel Guerrero-Zapata. An ANFIS-based Cache Replacement Method for Mitigating Cache Pollution Attacks in Named Data Networking. *Computer Networks*, 80:51–65, 2015.
- [14] Tobias Lauinger. Security & Scalability of Content-Centric Networking. Master’s thesis, Technische Universität Darmstadt, 2010.
- [15] Dapeng Man, Yongjia Mu, Jiafei Guo, Wu Yang, Jiguang Lv, and Wei Wang. Cache Pollution Detection Method Based on GBDT in Information-Centric Network. *Security and Communication Networks*, 2021(1), 2021.
- [16] Seyed Ali Mirheidari, Sajjad Arshad, Kaan Onarlioglu, Bruno Crispo, Engin Kirda, and William Robertson. Cached and Confused: Web Cache Deception in the Wild. In *USENIX Security Symposium*, 2020.
- [17] Seyed Ali Mirheidari, Matteo Golinelli, Kaan Onarlioglu, Engin Kirda, and Bruno Crispo. Web Cache Deception Escalates! In *USENIX Security Symposium*, 2022.
- [18] Mark Nottingham. The Cache-Status HTTP Response Header Field. RFC 9211, 2022. <https://datatracker.ietf.org/doc/html/rfc9211>.
- [19] Hyundo Park, Indra Widjaja, and Heejo Lee. Detection of Cache Pollution Attacks Using Randomness Checks. In *IEEE International Conference on Communications*, 2012.
- [20] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczynski, and Wouter Joosen. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In *Network and Distributed System Security Symposium*, 2019.
- [21] Alex Rousskov and Duane Wessels. High-performance Benchmarking with Web Polygraph. *Software: Practice and Experience*, 34(2):187–211, 2004.
- [22] Dave Smart and Jamie Indigo. Page Weight. The Web Almanac, 2024. <https://almanac.httparchive.org/en/2024/page-weight>.
- [23] Varnish. Hashing, 2025. <https://varnish-cache.org/docs/7.7/users-guide/vcl-hashing.html>.
- [24] Varnish. VCL - Varnish Configuration Language, 2025. <https://varnish-cache.org/docs/7.7/users-guide/vcl.html>.
- [25] Junwei Wang, Xianglin Wei, Jianhua Fan, Qiang Duan, Jianwei Liu, and Yangang Wang. Request Pattern Change-based Cache Pollution Attack Detection and Defense in Edge Computing. *Digital Communications and Networks*, 9(5):1212–1220, 2023.
- [26] Bastiaan Wissingh, Christopher A. Wood, Alex Afanasyev, Lixia Zhang, David R. Oran, and Christian Tschudin. Information-Centric Networking (ICN): Content-Centric Networking (CCNx) and Named Data Networking (NDN) Terminology. RFC 8793, 2020. <https://datatracker.ietf.org/doc/html/rfc8793>.
- [27] Mengjun Xie, Indra Widjaja, and Haining Wang. Enhancing Cache Robustness for Content-Centric Networking. In *IEEE International Conference on Computer Communications*, 2012.
- [28] xxHash. xxHash. <https://xxhash.com/>.
- [29] Lin Yao, Zhenzhen Fan, Jing Deng, Xin Fan, and Guowei Wu. Detection and Defense of Cache Pollution Attacks Using Clustering in Named Data Networks. *IEEE Transactions on Dependable and Secure Computing*, 17(6):1310–1321, 2020.
- [30] Lin Yao, Yujie Zeng, Xin Wang, Ailun Chen, and Guowei Wu. Detection and Defense of Cache Pollution Based on Popularity Prediction in Named Data Networking. *IEEE Transactions on Dependable and Secure Computing*, 18(6):2848–2860, 2021.

## A Ethical Considerations

All the attack experiments we present in this paper were conducted in realistic but controlled lab environments, without affecting external systems.

We also designed the Tranco top 10k experiments to minimize the traffic load on the targeted websites and to eliminate all damage outcomes. Specifically, we limited each crawl to at most 10 subdomains for each website and at most 10 pages per subdomain. We also limited our traffic rate to 1 request per second to avoid overwhelming the servers. To minimize the overhead of testing the websites for imprecise cache keys, we stopped on detecting the first successfully cache-busted object.

Most importantly, we note the following additional ethical considerations. The issue we present is a consequence of imprecise cache key configuration rather than a vendor-specific implementation vulnerability; consequently, disclosure to caching-proxy software vendors was not appropriate or likely to be actionable. More broadly, WCO reflects a configuration weakness, not a vendor-specific vulnerability. Cache key design is generally not treated as a security practice, and many operators may not explicitly manage cache keys, especially in managed, outsourced, or inherited deployments. While we identified imprecise cache keys, we did not validate the surrounding deployment context, such as whether a CDN was in use or whether other mitigations would make exploitation impractical. Reporting imprecise cache keys in isolation without validating exploitability risked creating confusion and overstating the practical severity for individual sites. We also did not perform vulnerability detection on production sites

that use caching proxies, which would have required active experiments capable of causing DoS. Conducting such testing would have been unethical and out of scope, even for sites that publish vulnerability disclosure policies, so we refrained from active probing. Therefore, we did not identify or report any specific vulnerable sites.

Finally, we did not issue targeted notifications or a separate public advisory because, to our knowledge, no CERT-like organization accepts advisories covering broad configuration-class issues of this scope, and targeted notification without deployment-specific validation could overstate risk rather than help remediation. Instead, we consider broad documentation of the issue together with the release of detection tooling in this paper to be the most responsible and viable approach for operators and researchers.

## B Generative AI Usage

Part of the code used in our experiments was developed with the assistance of AI-powered tools. These tools were employed to suggest code completions, boilerplate structures, and implementation details during development. All generated code was manually and thoroughly reviewed, tested, and adapted by the authors to ensure correctness and alignment with the intended experimental design.

## Sample GitHub Repositories

Table 9 lists a sample of the projects we detected on GitHub and analyzed for cache configurations. We present the top 10 projects for each caching proxy, sorted by popularity.

Table 9: Samples of the 10 most popular GitHub repositories where we identified a cache configuration for the 5 selected proxies.

| Cache   | Repository Name                              | Stars | Size (MB) |
|---------|----------------------------------------------|-------|-----------|
| ATS     | apache/trafficserver                         | 1860  | 256       |
| ATS     | mustafaramadhan/kloxo                        | 336   | 256       |
| ATS     | sqawasm/trafficserver-docker                 | 19    | 256       |
| ATS     | Li4n0/My-CTF-Challenges                      | 16    | 256       |
| ATS     | sunnyszy/lrb-prototype                       | 9     | 1048576   |
| ATS     | ShufanWangBGM/Reinforcement- [...]           | 8     | 524288    |
| ATS     | xyp-root/geektime-hands-on- [...]            | 3     | 1024      |
| ATS     | JasonGiedymin/ats-docker                     | 2     | 256       |
| ATS     | mingzym/zym4T                                | 2     | 144       |
| ATS     | bryancall/benchmark                          | 1     | 102400    |
| HAProxy | haproxy/haproxy                              | 5508  | 200.0     |
| HAProxy | haproxytech/dataplaneapi                     | 345   | 1024.0    |
| HAProxy | haproxytech/client-native                    | 133   | 4.0       |
| HAProxy | http-tests/cache-tests                       | 122   | 4.0       |
| HAProxy | haproxytech/config-parser                    | 82    | 4.0       |
| HAProxy | intel/workload-services-framework            | 55    | 128.0     |
| HAProxy | existentialcomics/kungFuChess                | 31    | 200.0     |
| HAProxy | xen0bit/muvr.xyz                             | 22    | 64.0      |
| HAProxy | zairo-korea/http_request_smuggling_test      | 18    | 4.0       |
| HAProxy | haproxytech/haproxy-dev-lua-filters          | 5     | 200.0     |
| NGINX   | Gallopsled/pwntools                          | 12518 | 1024.0    |
| NGINX   | ShaneIsrael/freshshare                       | 757   | 500.0     |
| NGINX   | huasenjio/huasenjio-compose                  | 560   | 51200.0   |
| NGINX   | aldor007/mort                                | 513   | 73.0      |
| NGINX   | PUGX/badge-poser                             | 477   | 500.0     |
| NGINX   | brunobritodev/JPPProject.IdentityServer4.SSO | 455   | 1024.0    |
| NGINX   | dyc3/opentogethertube                        | 427   | 1000.0    |
| NGINX   | DanWahlin/Angular-Docker-Microservices       | 220   | 3000.0    |
| NGINX   | yuri-gushin/Roboo                            | 180   | 1000.0    |
| NGINX   | PencilCode/pencilcode                        | 168   | 200.0     |
| Squid   | bannedbook/fanqiang                          | 39502 | 5000.0    |
| Squid   | vimagick/dockerfiles                         | 3181  | 100.0     |
| Squid   | av/harbor                                    | 1576  | 100.0     |
| Squid   | xjdrew/kone                                  | 705   | 10240.0   |
| Squid   | diladele/squid-windows                       | 194   | 100.0     |
| Squid   | salrashid123/squid_proxy                     | 128   | 100.0     |
| Squid   | 1265578519/PAC                               | 54    | 5000.0    |
| Squid   | jacobproject/operation                       | 40    | 800.0     |
| Squid   | CSCfi/ansible-role-squid                     | 9     | 100.0     |
| Squid   | linuxgazette/lg                              | 5     | 20480.0   |
| Varnish | varnish/Varnish-Book                         | 353   | 256.0     |
| Varnish | wenerme/wener                                | 300   | 32768.0   |
| Varnish | aliuosio/mage2.docker                        | 58    | 2048.0    |
| Varnish | aqzt/docker-alpine                           | 53    | 100.0     |
| Varnish | localwiki/localwiki-backend-server           | 48    | 712.0     |
| Varnish | camptocamp/puppet-varnish                    | 36    | 256.0     |
| Varnish | roadiz/skeleton                              | 11    | 256.0     |
| Varnish | silentred/learning-path                      | 8     | 256.0     |
| Varnish | waldronlab/BugSigDB                          | 8     | 100.0     |
| Varnish | waldronlab/BugSigDB                          | 8     | 100.0     |